

CX4240 Spring 2026

Convolution Neural Networks

Bo Dai
School of CSE, Georgia Tech
bodai@cc.gatech.edu

Vector Formation

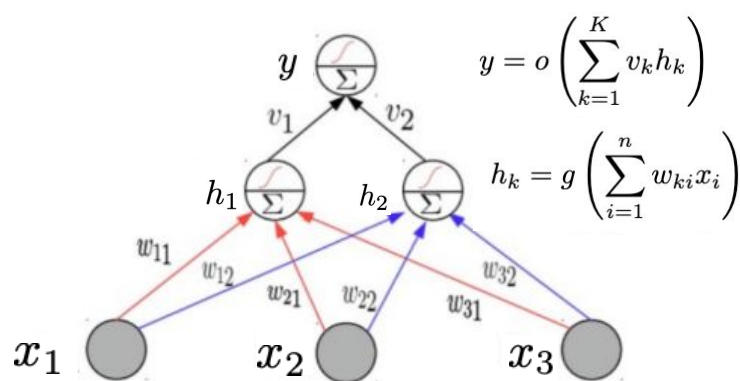
$$y = o(\mathbf{V}g(\mathbf{W}x))$$

$$\mathbf{V} = [v_1, v_2]$$

$$\mathbf{h} = [h_1, h_2]^\top = g(\mathbf{W}x)$$

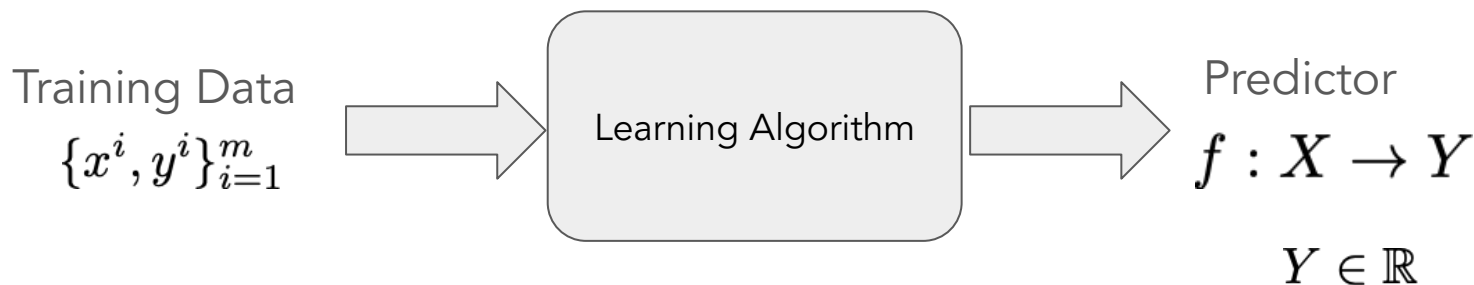
$$\mathbf{W} = \begin{bmatrix} w_{11} & w_{21} & w_{31} \\ w_{12} & w_{22} & w_{32} \end{bmatrix}$$

$$\mathbf{x} = [x_1, x_2, x_3]^\top$$



Backward Pass

Regression Algorithms

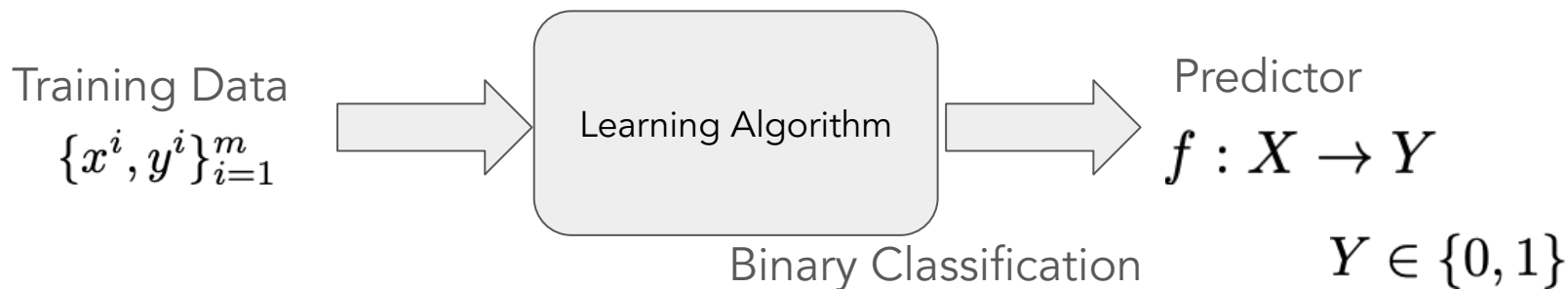


Linear Regression Pipeline

1. Build probabilistic models:
Gaussian Distribution + Neural Network
2. Derive loss function: MLE and MAP
3. Select optimizer: (Stochastic) GD

$$y = o(\mathbf{V} g(\mathbf{W} x))$$

Binary Classification Algorithms



Binary Logistic Regression Pipeline

1. Build probabilistic models:

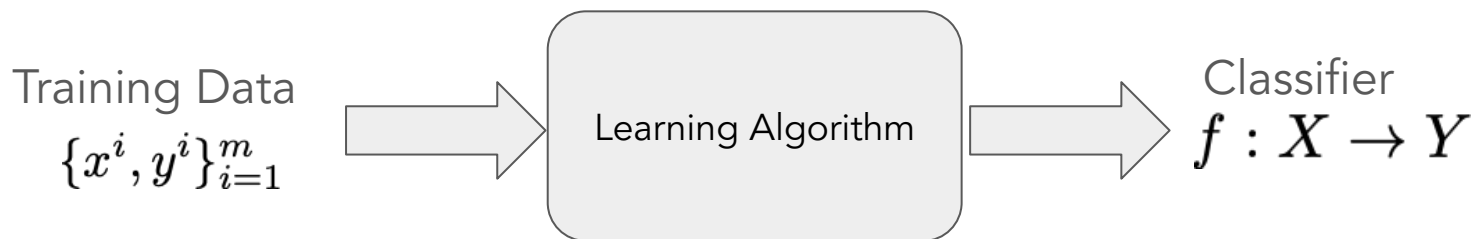
Bernoulli Distribution + Neural Network

2. Derive loss function: MLE and MAP

3. Select optimizer: (Stochastic) Gradient Descent

$$y = o(\mathbf{V} g(\mathbf{W} x))$$

Multiclass Logistic Regression Algorithms



Multiclass Classification $Y \in \{0, 1, \dots, k\}$
Multiclass Logistic Regression Pipeline

1. Build probabilistic models:
Categorical Distribution + Neural Network $y = o(\mathbf{V} g(\mathbf{W} x))$
2. Derive loss function: MLE and MAP
3. Select optimizer: (Stochastic) Gradient Descent

Select Optimizer

$$L(\theta) = \sum_{i=1}^m \ell(x^i, y^i, \theta) + \lambda \Omega(\theta)$$
$$\theta = [V, W]$$

$$\ell(x^i, y^i, \theta) = (o(Vg(Wx^i)) - y^i)^2$$

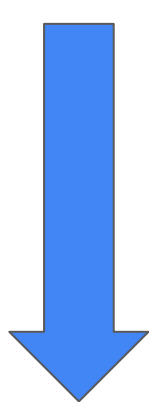
$$\ell(x^i, y^i, \theta) = -y^i \log \sigma(o(Vg(Wx^i)))$$
$$-(1 - y^i) \log(1 - \sigma(o(Vg(Wx^i))))$$

$$\ell(x^i, y^i, \theta) = - \sum_{j=1}^k y^i \log \frac{\exp(o(V_j g(Wx^i)))}{\sum_{c=1}^k \exp(o(V_c g(Wx^i)))}$$

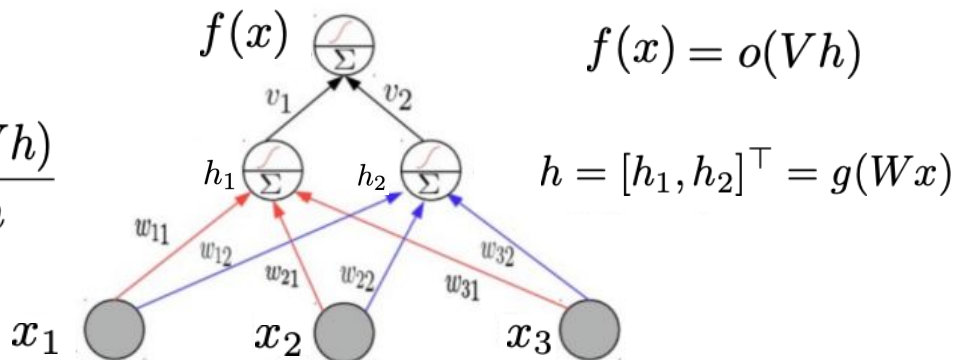
- (Stochastic) Gradient Descent

Backpropagation: Chain Rule on Neural Network

$$\ell(x^i, y^i, \theta) = (f(x^i, \mathbf{V}, \mathbf{W}) - y^i)^2$$



$$\frac{\partial \ell(x^i, y^i, \theta)}{\partial f}$$
$$\frac{\partial o(Vh)}{\partial V} \cdot \frac{\partial o(Vh)}{\partial h}$$
$$\frac{\partial h}{\partial W}$$



Backward Pass

(Stochastic) Gradient Descent

- Initialize parameter θ^0
- Sample $\{x^i, y^i\}_{i=1}^B$
- Do
$$\theta^{t+1} \leftarrow \theta^t - \eta \sum_{i=1}^B \nabla_{\theta} \ell(x^i, y^i, \theta^t) - (\lambda \nabla \Omega(\theta^t))$$

Auto-differentiation Packages

PyTorch



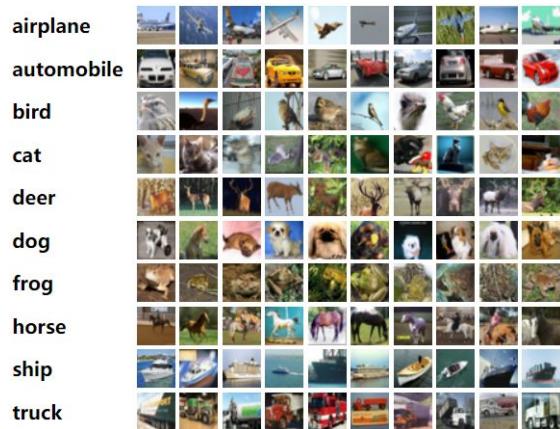
JAX



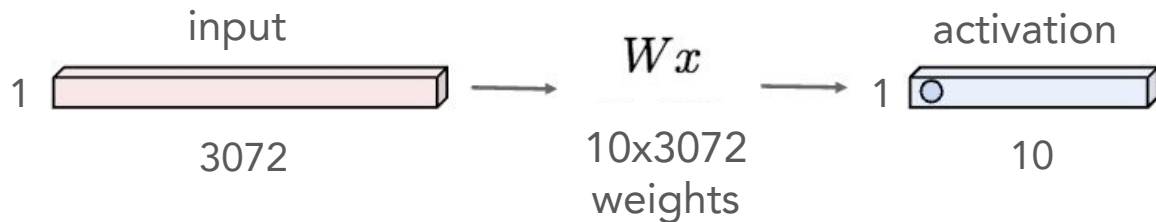
Tensorflow



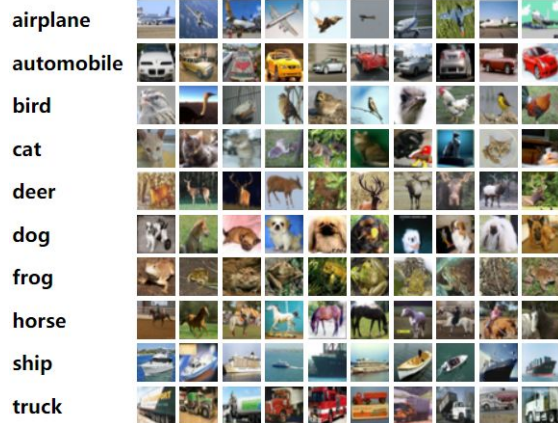
Fully Connected Layer



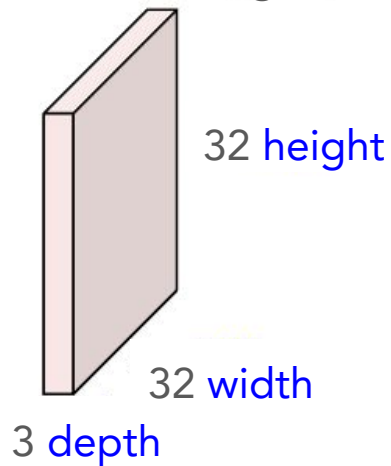
32x32x3 image $\rightarrow$ stretch to 3072x1



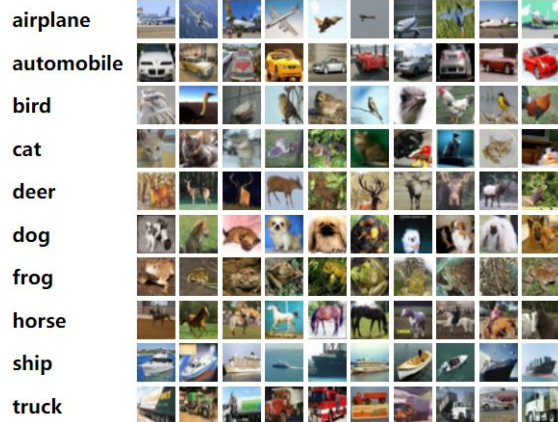
Convolution Layer



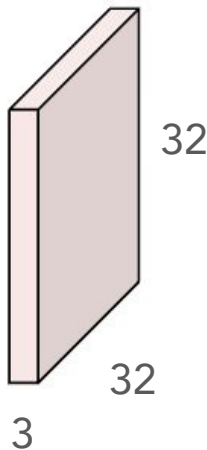
32x32x3 image → preserve spatial structure



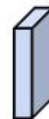
Convolution Layer



32x32x3 image

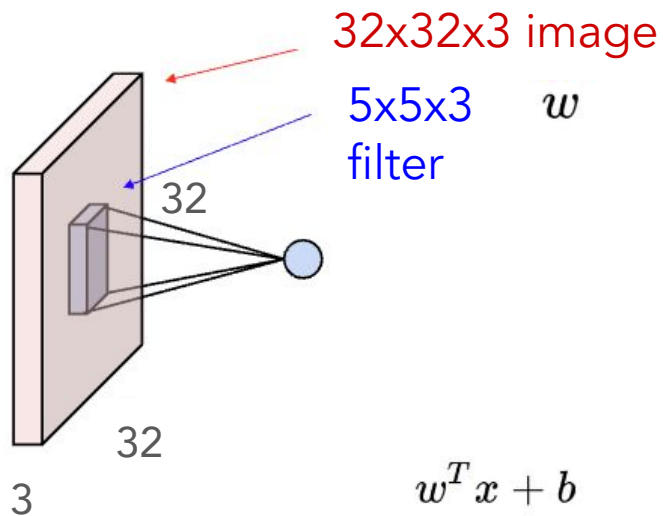
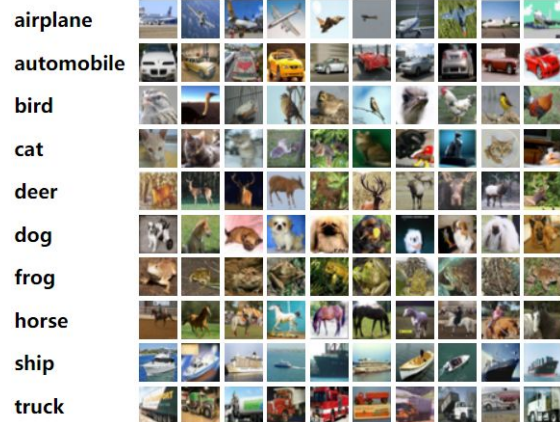


5x5x3 filter

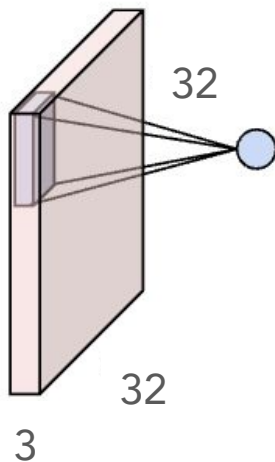
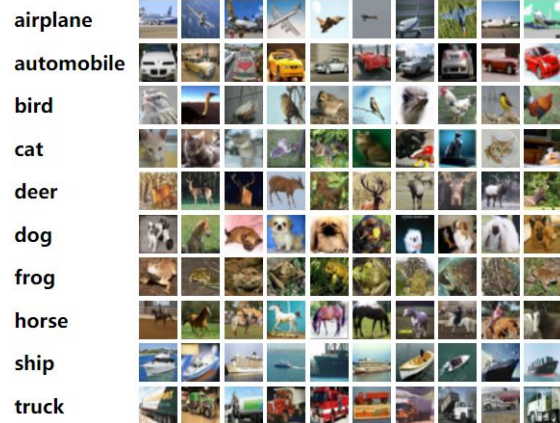


Convolve the filter with the image,
i.e., "slide over the image spatially,
computing dot product"

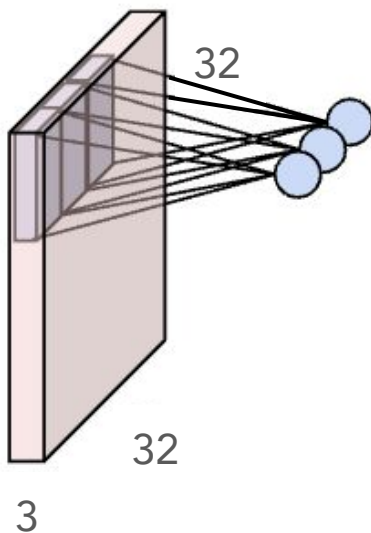
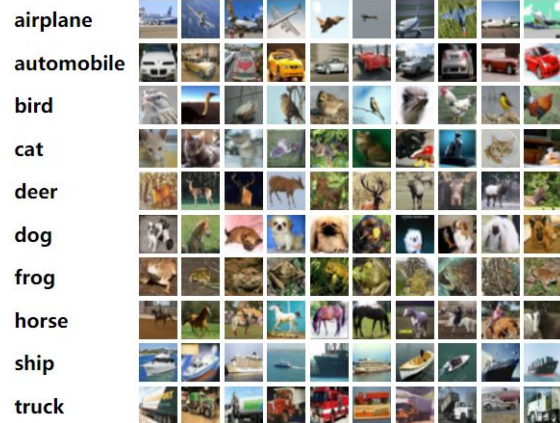
Convolution Layer



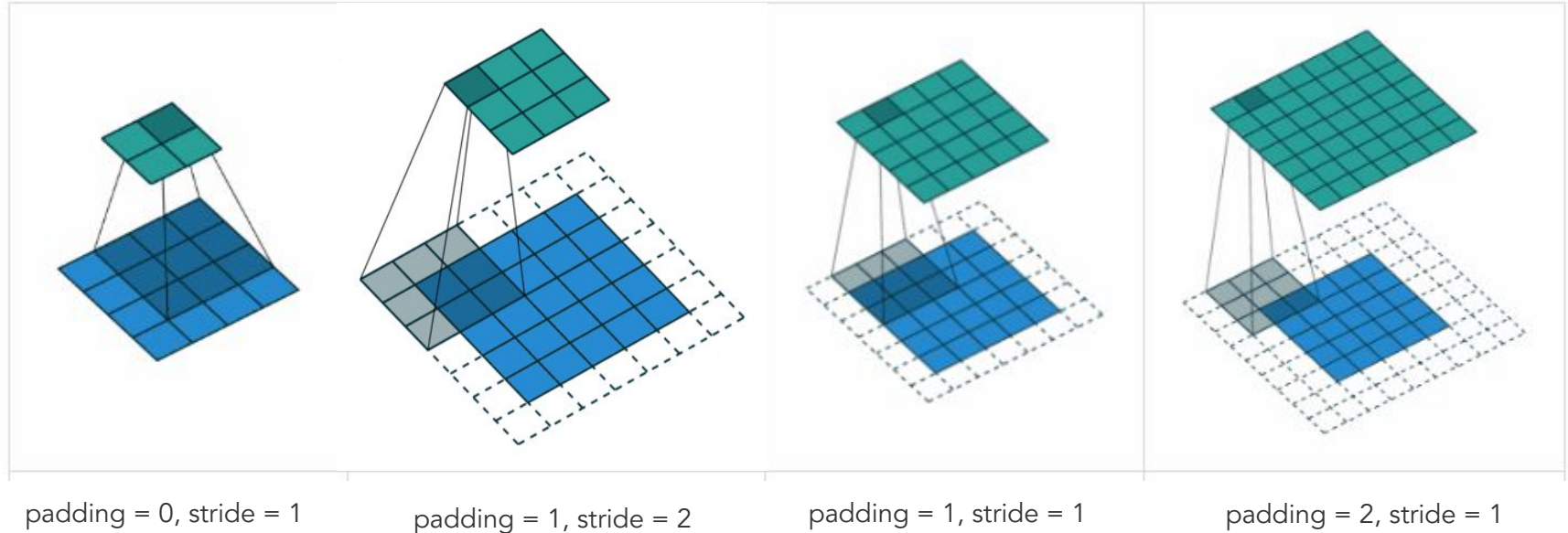
Convolution Layer



Convolution Layer



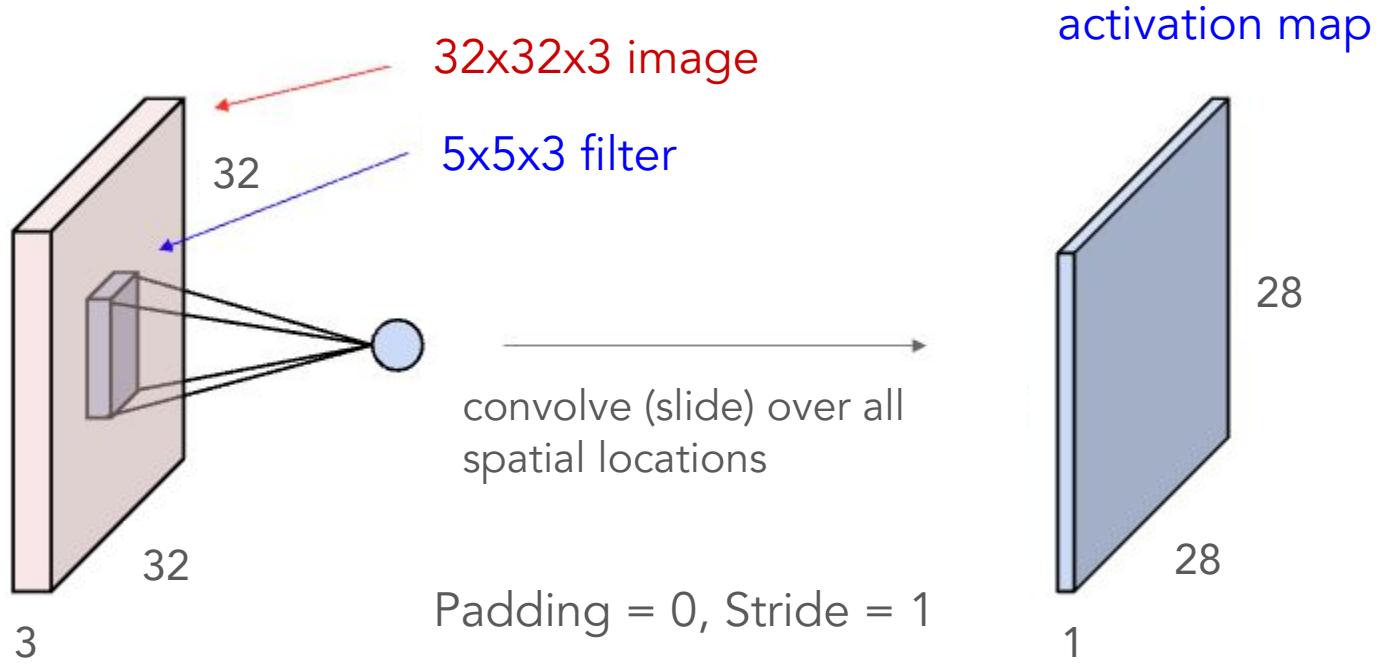
Convolution Layer



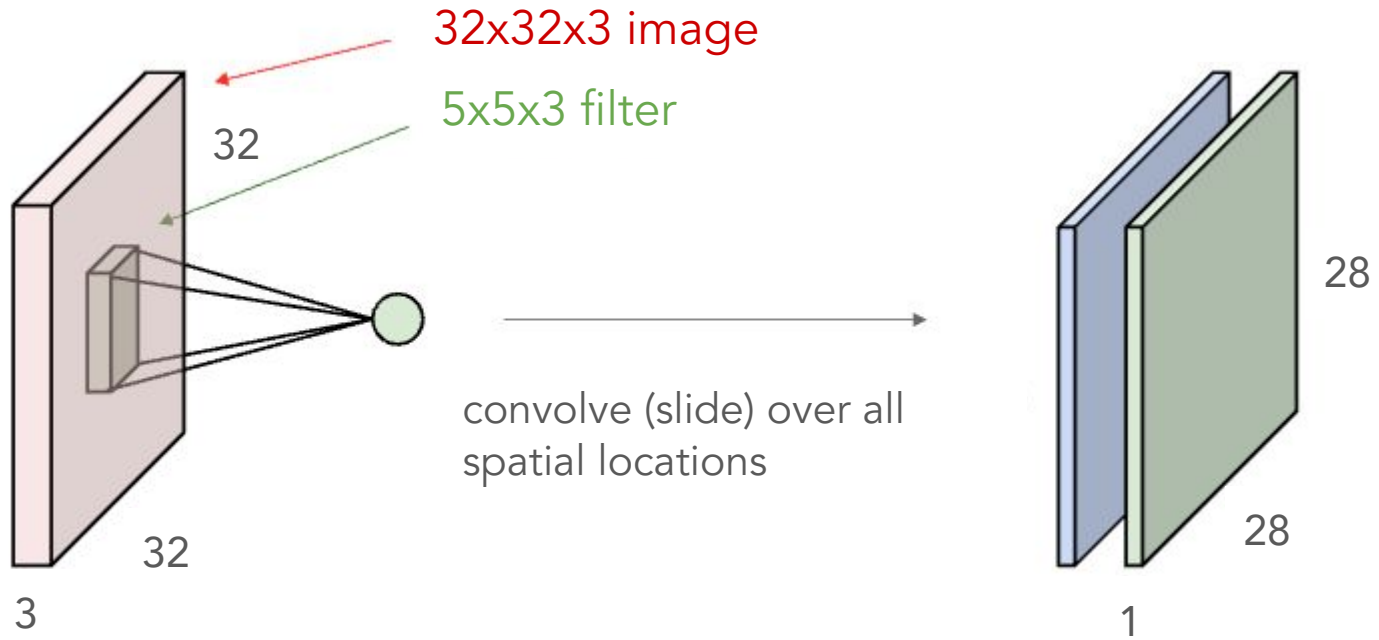
$$W_{out} = \frac{W - F + 2P}{S} + 1$$

Animation from [Hochschule der Medien](#)

Convolution Layer

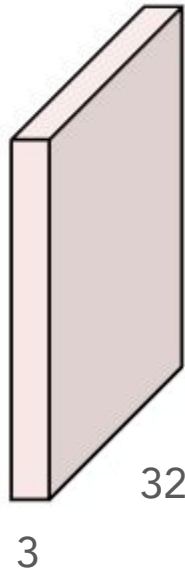


Convolution Layer

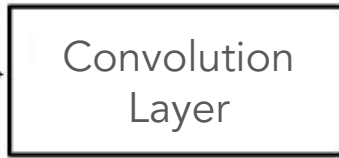


Convolution Layer

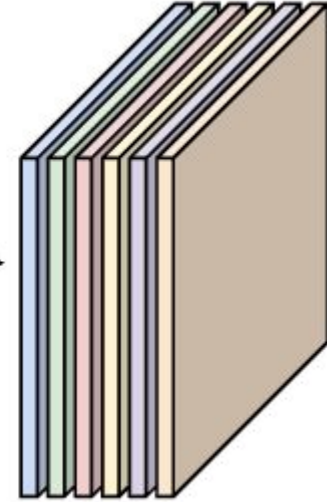
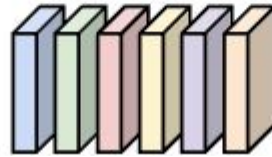
32x32x3 image



Consider 6 filters,
each 3x5x5



6x3x5x5
filters

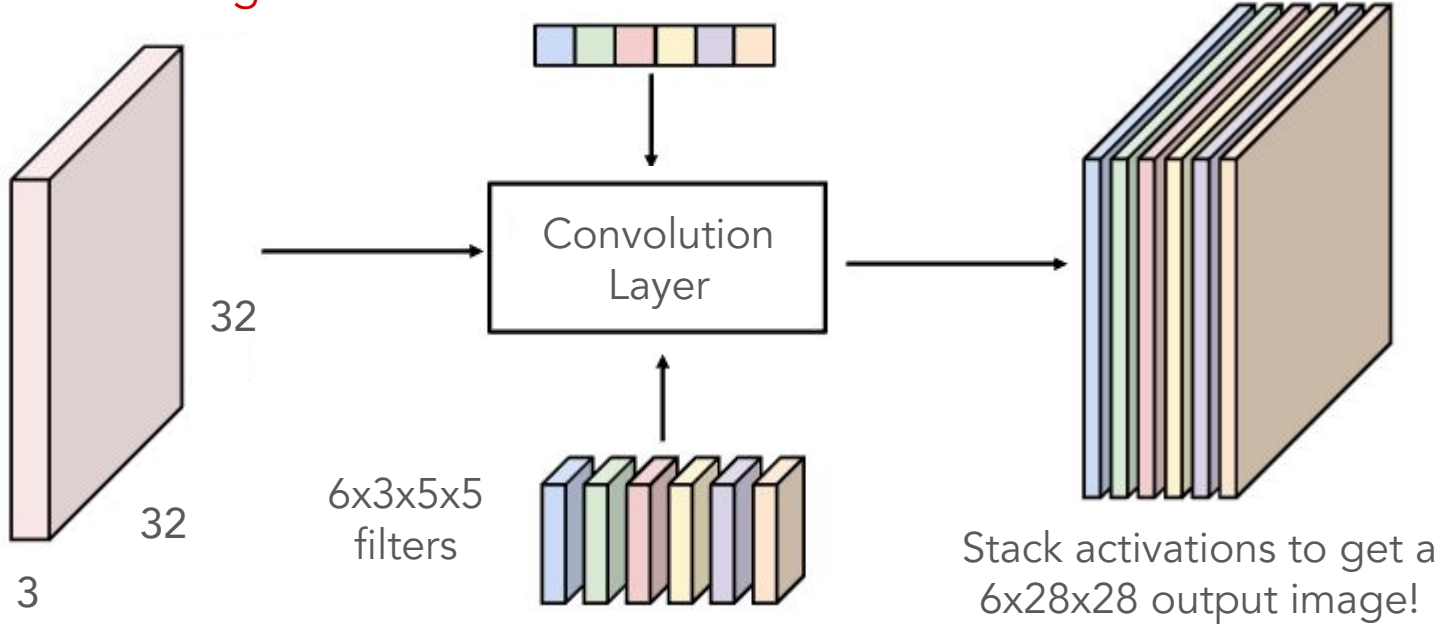


Stack activations to get a
6x28x28 output image!

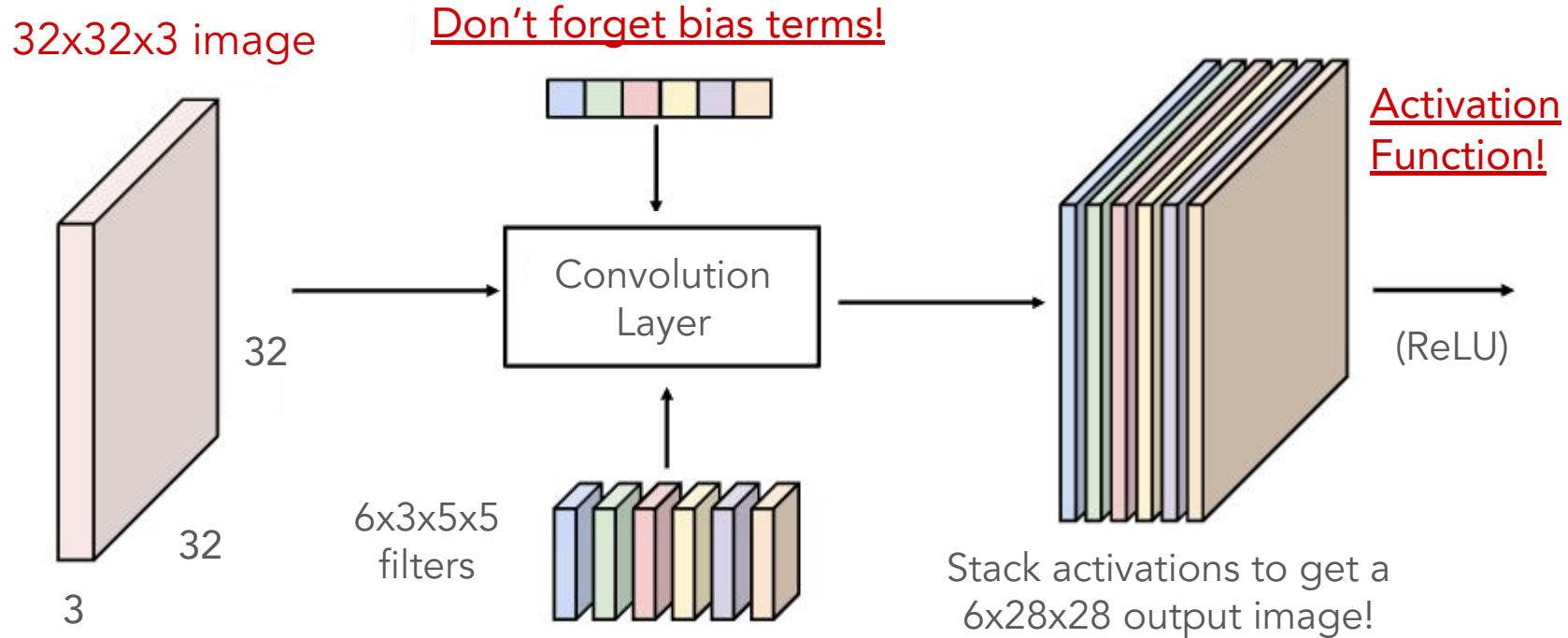
Convolution Layer

32x32x3 image

Don't forget bias terms!

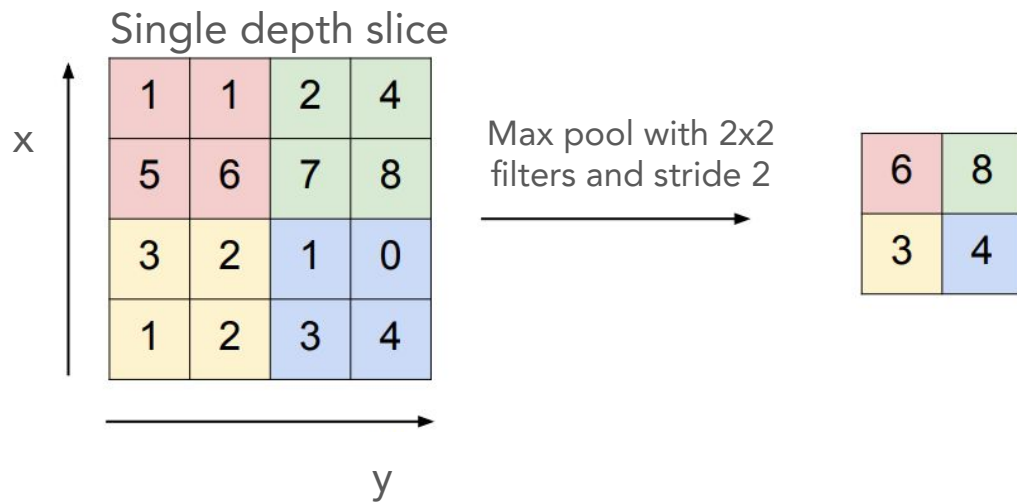


Convolution Layer



Pooling (Subsampling)

MAX POOLING

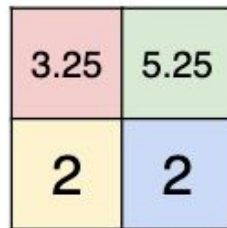
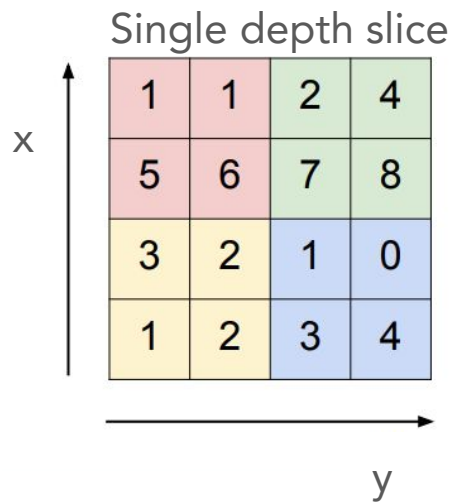


$$(W_x - F) / \text{Stride} + 1$$

$$(W_y - F) / \text{Stride} + 1$$

Pooling (Subsampling)

MEAN POOLING

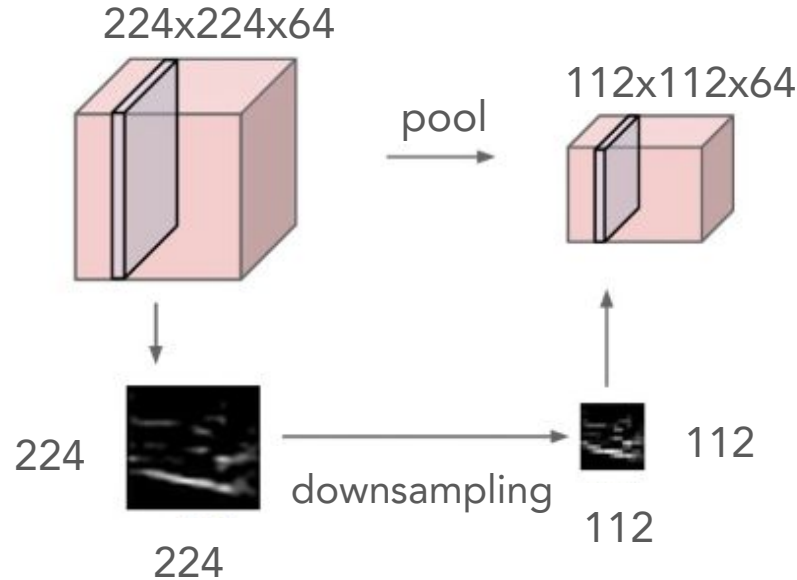


$$(W_x - F) / \text{Stride} + 1$$

$$(W_y - F) / \text{Stride} + 1$$

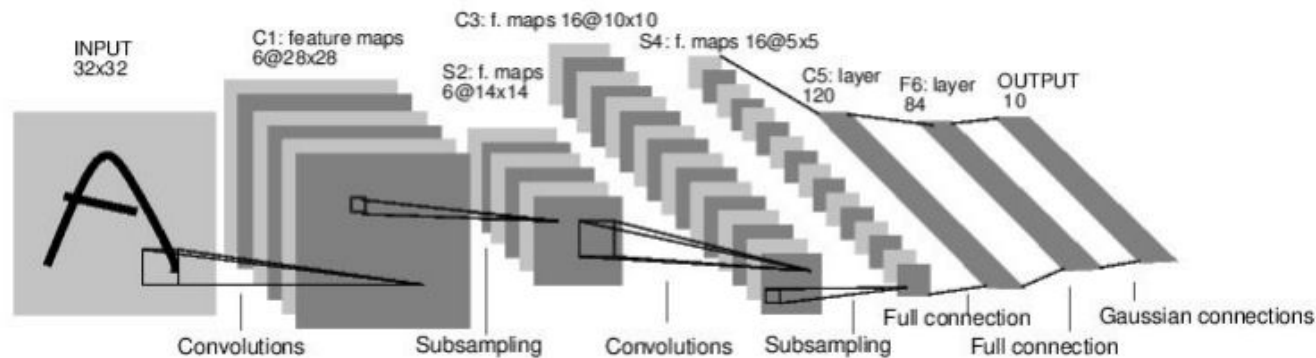
Average
Pooling

Pooling (Subsampling)



- Pooling layers simplify / subsample / compress the information in the output from the convolutional layer
- Reduce parameters

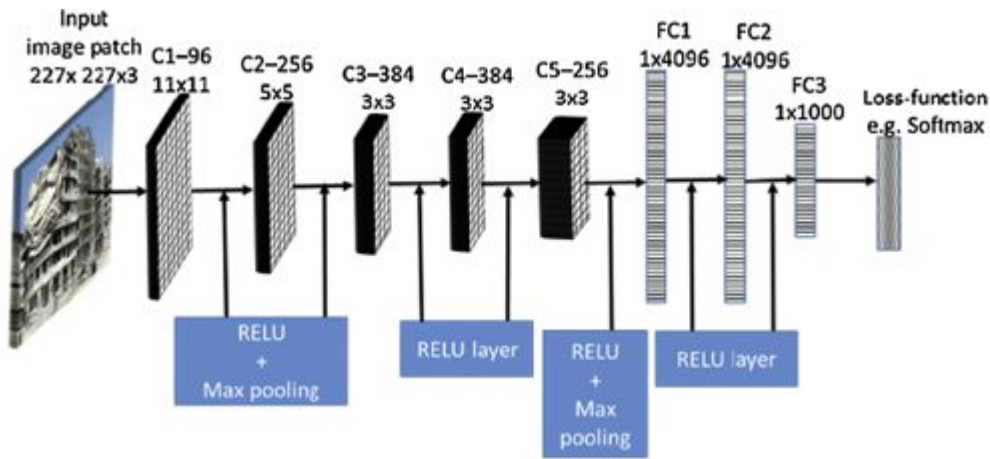
Put Everything Together



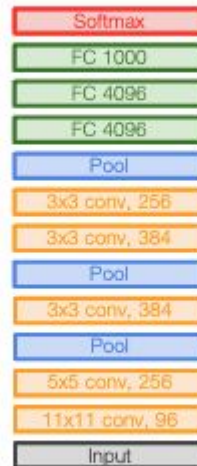
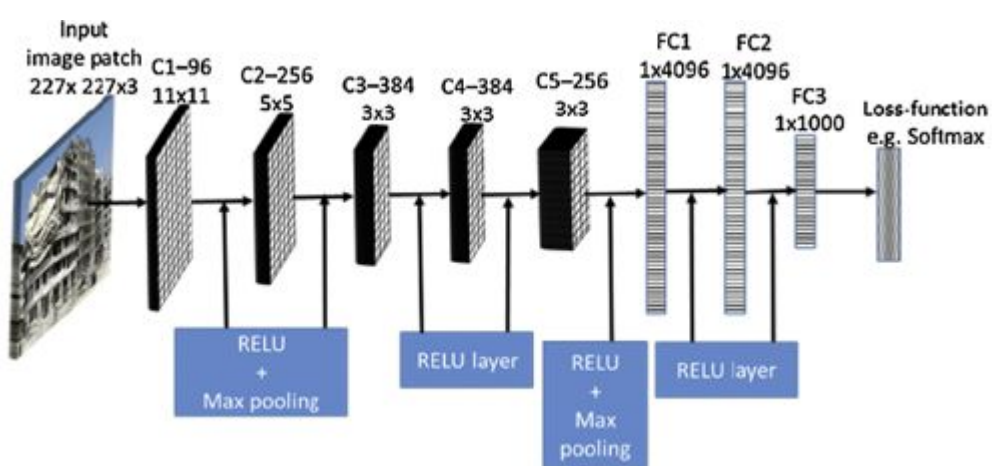
[LeNet-5, LeCun 1980]

AlexNet (2012)

- AlexNet was one of the first deep convolutional on ImageNet competition with an accuracy of **84.7%** as compared to the second-best with an accuracy of **73.8%**.
- The activation used is the Rectified Linear Unit (ReLU).
- The training set had 1.2 million images. It was trained for 90 epochs, which took **five to six days** on two NVIDIA GTX 580 3GB GPUs.



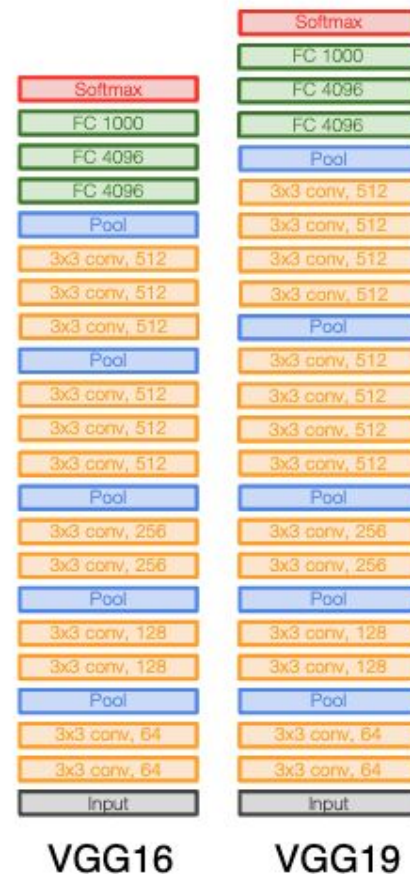
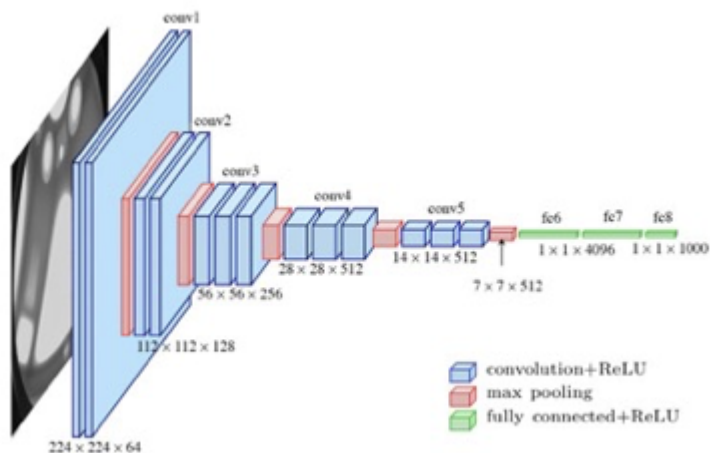
AlexNet (2012)



AlexNet

VGGNet (2014)

- Very Deep CNN
- With only 3*3 conv filters
 - Fewer parameters, deeper nonlinear layers



ResNet (2015)

- Very Deep CNN with residual connections
 - 152-layer model for ImageNet
 - ILSVRC'15 classification winner (3.57% top 5 error)
 - Swept all classification and detection competitions in ILSVRC'15 and COCO'15!

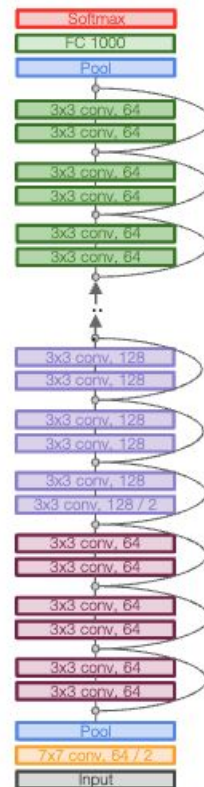
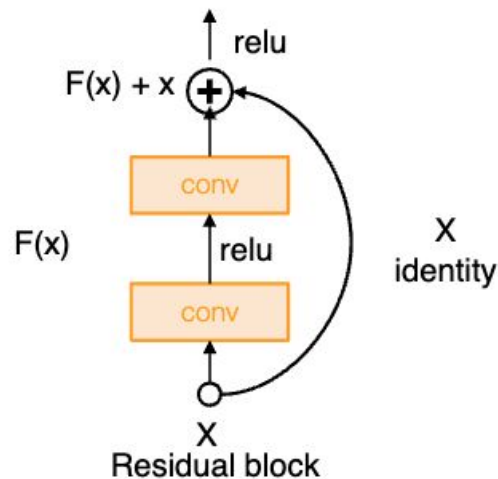


Image Classification on ImageNet

Leaderboard

Community Models

Dataset

View

Top 1 Accuracy



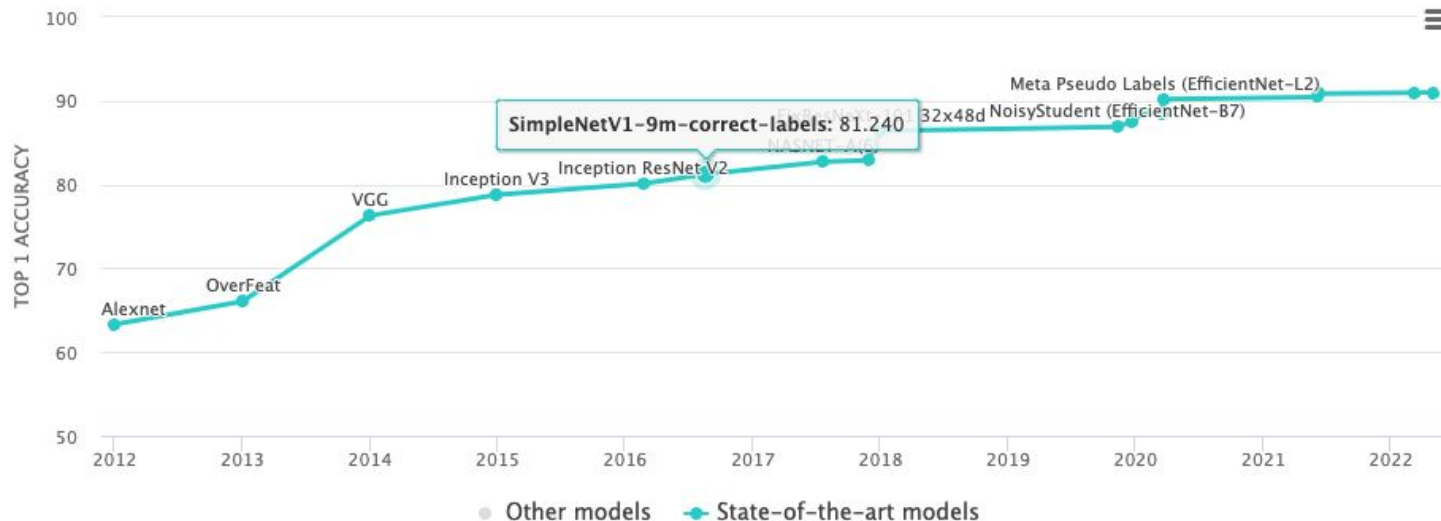
by

Date

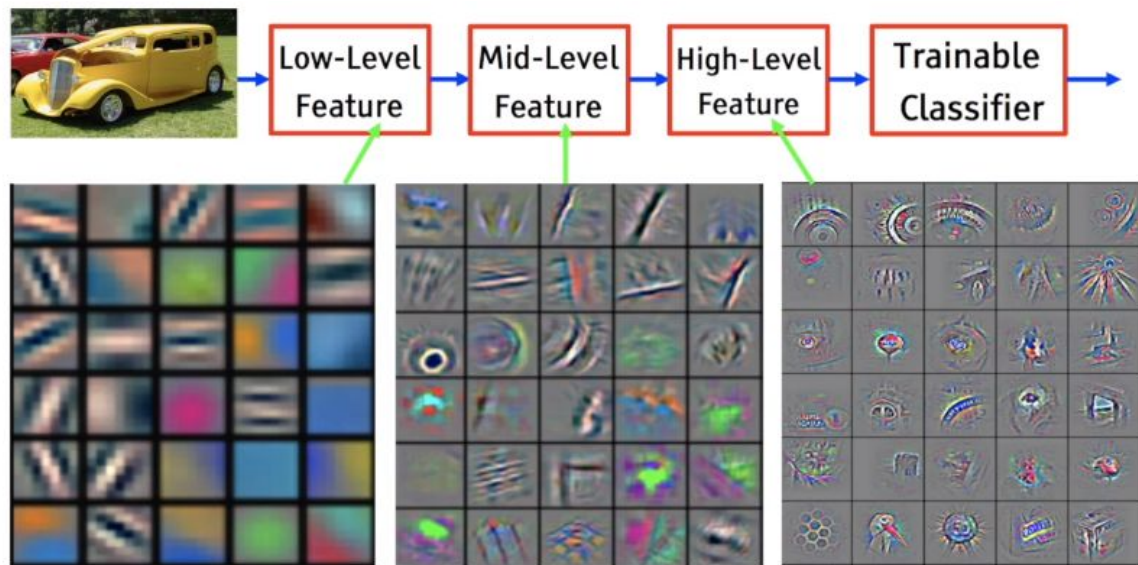


for

All models



Convolution Features



VGG16

VGG19

Auto-differentiation Packages

PyTorch



JAX

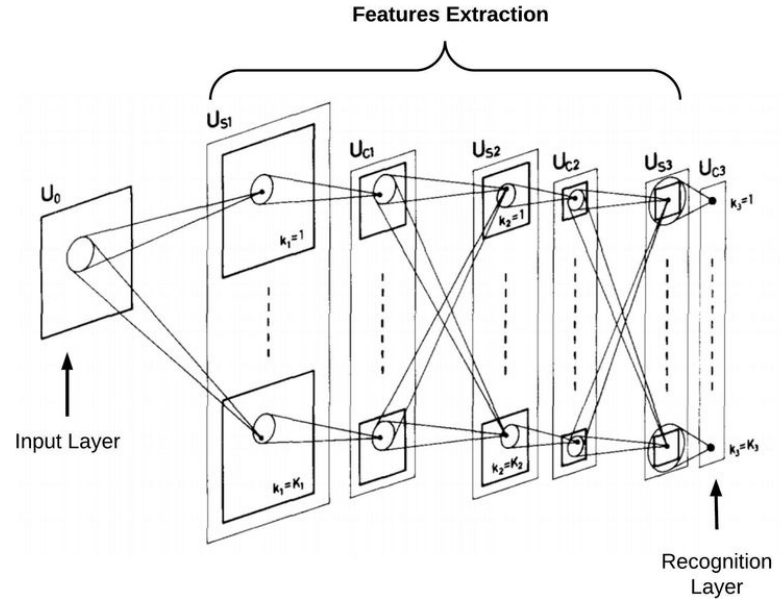


Tensorflow



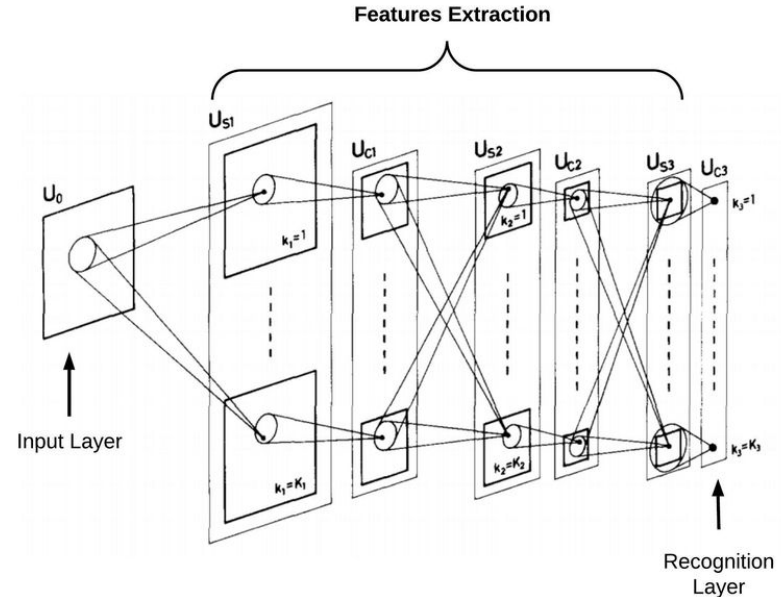
A Bit of History

- [Neocognitron](#) [Fukushima 1980]



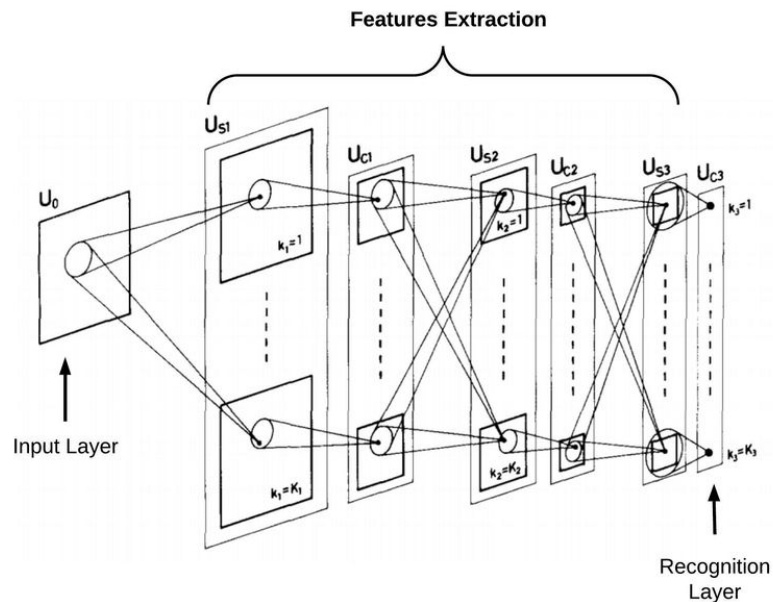
A Bit of History

- Neocognitron [[Fukushima 1980](#)]
- Backpropagation [[Rumelhart et al., 1986](#), [Werbos 1974](#)]



A Bit of History

- Neocognitron [[Fukushima 1980](#)]
- Backpropagation [[Rumelhart et al., 1986](#), [Werbos 1974](#)]
- BP on CNN [[LeCun et al., 1989](#)]



Q&A

HW2 Due Today!

No HW until Mar. 05th
Use the time wisely for project